

DeepSolo-inspired Ordered-Point Hangul OCR: Korean Reading-Order Correction for Apple Vision OCR and the HangulGuidedOCR Library

Minji Jeong

Team 27

Abstract. This study aims to improve reading-order errors and text omissions in Apple Vision OCR for real-world Korean images such as posters, signboards, and subway notices, where characters are arranged vertically or scattered across the layout. We analyze DeepSolo’s ordered-points and boundary-based text spotting representation. Although official DeepSolo inference could not be fully reproduced because the available environment was macOS arm64 without the CUDA/Detectron2 stack required by the original implementation, we reproduced its representation idea through a proxy that converts OCR bounding boxes into 25 ordered center-line points and boundary information. Inspired by this representation, we designed HangulGuidedOCR(auto), a correction layer for Apple Vision OCR, and compared raw Vision, preprocessing-only, postprocessing-only, and preprocessing plus postprocessing auto conditions. Following instructor feedback, Apple Vision OCR is reframed as a closed-source black-box baseline whose internal detector/recognizer cannot be isolated, and Tesseract, EasyOCR, and PaddleOCR are added as public OCR baselines. On the Main GT aggregate, raw Vision OCR produced a CER of 1.209, whereas HangulGuidedOCR(auto) reduced it to 0.018. The effect was especially clear for vertical station signs, dispersed typography, and stylized title-omission cases.

Keywords: DeepSolo; Apple Vision OCR; Korean OCR; scene text spotting; reading order; Hangul-GuidedOCR

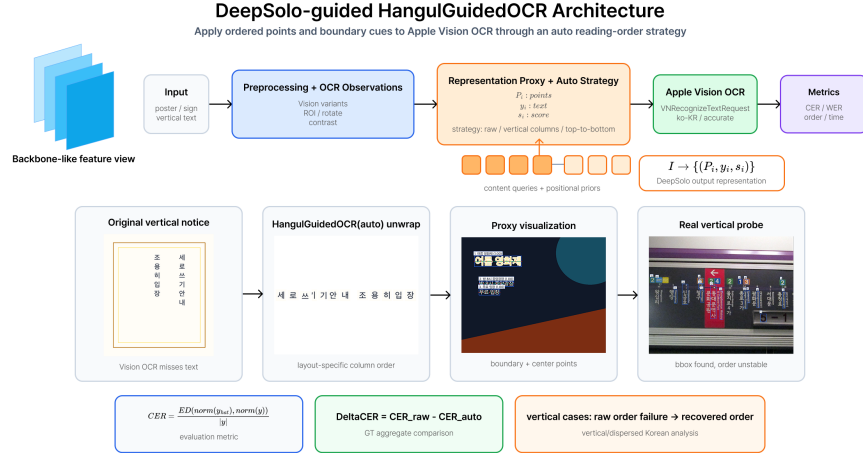


Fig. 1: HangulGuidedOCR architecture, which adapts DeepSolo-inspired ordered-point representation into an Apple Vision OCR correction pipeline.

1 Introduction

OCR is a core capability for mobile applications that convert camera input into text, but Korean posters, signboards, and notices are more complex than ordinary document OCR. In subway station signs, vertically written station names require a semantically meaningful reading order. In poster-style dispersed typography, words often do not follow a standard line structure. Apple Vision OCR is strong on controlled horizontal

text, but in these layouts it does not always guarantee the string order expected by Korean readers, even when text observations are detected.

The goal of this project is not to port the DeepSolo model itself to macOS. Instead, it transfers the text-instance representation proposed by DeepSolo, namely ordered points and text boundaries, into input correction and output postprocessing for Apple Vision OCR. The study asks three questions. First, how far can the official DeepSolo baseline be reproduced in the current Mac environment? Second, can a proxy reproduction of the DeepSolo representation explain and correct Korean reading-order failures in Apple Vision OCR? Third, can this correction strategy be generalized into a Swift/Xcode library API usable by application developers? The contributions are therefore: treating Apple Vision OCR as a black-box baseline, implementing a reproducible DeepSolo-inspired ordered-point proxy, and evaluating its effect with CER/WER and representative failure cases.

2 Related Work

Scene text spotting has evolved from RoI-based systems that separate detection and recognition, through segmentation-based systems, to Transformer-based end-to-end systems. CRNN [8] and ASTER [9] are widely used scene text recognition baselines, while TextBoxes [3], EAST [14], CRAFT [1], and DB [4] represent major text detection approaches. ABCNet [5], TESTR [13], and DeepSolo [12] directly address curved text and end-to-end text spotting.

From the OCR-engine perspective, systems such as Apple Vision OCR, Tesseract [10], EasyOCR [2], PaddleOCR [7], and Naver CLOVA OCR [6] perform well on document-like or horizontal text, but they do not always preserve the reading order expected in Korean vertical writing, multi-column signs, or dispersed typography. LayoutReader [11] treats reading order as a layout-aware ordering problem rather than a simple left-to-right connection problem. This study does not modify Apple Vision OCR internally and does not assume that DeepSolo is part of Apple Vision. Apple Vision OCR is used as a black-box baseline, while DeepSolo is used as a representational reference from end-to-end text spotting research. In the added comparison, Tesseract, EasyOCR, and PaddleOCR are not used to determine a final product winner. They are public baselines for testing whether ordered strips and pre-spot proposals improve the input structure across OCR engines.

3 Proposed Method

3.1 DeepSolo Reproduction and Proxy

DeepSolo represents text instances through explicit point queries and point priors in a Transformer encoder-decoder structure. In the paper, the output for an image I can be summarized as $\{(P_i, y_i, s_i)\}$, where P_i is interpreted as points connected to the text centerline/boundary, y_i is the recognized string, and s_i is the confidence score. The official GitHub implementation depends heavily on Detectron2, CUDA, and specific PyTorch/CUDA versions. Because the experimental environment was macOS arm64, direct CUDA inference was not possible, and Docker/UTM reproduction also stopped before model inference because of dependency resolution and the absence of GPU acceleration.

The reproduction is therefore divided into two levels. The first is official reproduction, where the demo entry point and Docker runbook were inspected and failure points were recorded. The second is proxy reproduction, where Apple Vision observations are transformed into 25 ordered center-line points and a boundary polygon. This proxy is not DeepSolo’s learned point query, so it cannot be claimed as a reproduction of DeepSolo model performance. Instead, it is a structural reproduction that explains why a DeepSolo-inspired representation can be useful for correcting Apple Vision OCR.

Table 1: Scope of DeepSolo reproduction and proxy conversion.

Item	Result	Interpretation
Official inference	Stopped	CUDA/Detectron2/Linux dependency mismatch
Proxy representation	Completed	Vision bbox $\rightarrow$ 25 ordered points + boundary polygon
Claim level	Representation	Not a DeepSolo model-performance reproduction

3.2 Overall Pipeline

The full pipeline consists of input image $\rightarrow$ ROI detection/crop $\rightarrow$ rotation correction $\rightarrow$ contrast/sharpness/upscaling $\rightarrow$ Apple Vision OCR $\rightarrow$ ordered-point postprocessing $\rightarrow$ CER/WER evaluation. At the input stage, we evaluated not only full-image OCR but also ROI crop, rotate, contrast, and upscale2 variants. At the representation-proxy stage, OCR observations are converted into ordered points and boundaries, and the auto strategy decides whether to preserve the raw order or apply reordering.

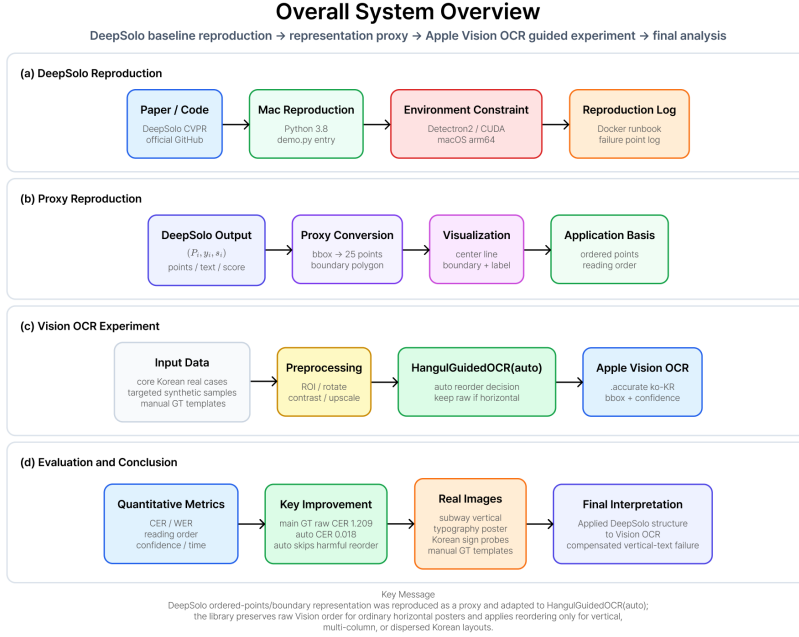


Fig. 2: Overall research flow from DeepSolo reproduction to representation proxy, Apple Vision OCR correction experiments, and final analysis.

3.3 Ordered Point Conversion

Given an Apple Vision observation $o_i = (B_i, y_i, s_i)$, let the four corners of B_i be $b_{tl}, b_{tr}, b_{br}, b_{bl}$. The two endpoints of the centerline are defined as $c_0 = (b_{tl} + b_{bl})/2$ and $c_1 = (b_{tr} + b_{br})/2$. The $N = 25$ ordered center-line points are sampled as

$$p_k = (1 - t_k)c_0 + t_k c_1, \quad t_k = \frac{k}{N-1}, \quad k = 0, \dots, N-1. \quad (1)$$

This point proxy is grouped and sorted by one of four strategies: horizontal-lines, vertical-columns-left-to-right, vertical-columns-right-to-left, or top-to-bottom. Overlapping proposals are reduced by overlap suppression, and repeated point characters are collapsed through CTC-like decoding.

3.4 Pre-OCR Proposal and Auto Strategy

The pre-OCR spot proposal is designed for cases where raw Vision misses a large title or vertical column. It first proposes a likely text region and then runs Vision OCR on that region. The proposal combines whole-image OCR evidence, high edge/contrast ROIs, and expected vocabulary hints supplied by the user or application domain. Vertical candidates are OCRed again after a 90° rotation, while poster-title candidates add crop and upscale variants. The auto strategy then selects one of raw, vertical L→R, vertical R→L, or top-to-bottom.

Table 2: Internal decision rules of HangulGuidedOCR(auto) by layout type.

Case	Auto action	Expected output
Horizontal poster	Keep raw order	Preserve raw-level CER
Subway station	Rotate + expectedOrder	Dongdaemun History Culture Park
Top-to-bottom typography	Row sort + vocabulary	Recover intended Korean title
Known domain terms	Fuzzy correction	Recover app-domain vocabulary

HangulGuidedOCR is always called as an OCR wrapper, but its internal auto strategy decides the layout. If there is one or no observation, raw order is preserved. If two or more tall point sequences have height more than twice their width and an x-center spread greater than 0.10, the layout is treated as a vertical-column candidate. These thresholds are not learned optimums; they are pilot defaults chosen to avoid damaging ordinary horizontal posters. In the manual-template set, forced proxy reordering increased CER to 0.191 compared with raw/auto CER of 0.060, demonstrating why an auto gate is required.

3.5 Program Code

The implementation of HangulGuidedOCR is divided into three modules: a Vision OCR wrapper, a point-proxy builder, and a reading-order resolver. The Vision wrapper collects the string, confidence, and normalized bounding box returned by Apple Vision. The point-proxy builder converts each box into 25 ordered center-line points and a boundary polygon. The reading-order resolver uses the aspect ratio, column spread, and row-grouping evidence of the point sequence to determine whether to preserve raw order or reorder it as vertical columns or top-to-bottom text. Listing 1.1 shows the core logic. For ordinary horizontal paragraphs, `rawVisionOrder` is returned; ordered-point reordering is applied only for vertical or dispersed layouts.

Listing 1.1: Core Swift implementation excerpt from HangulGuidedOCR.

```
public static func orderedPoints(
    for observation: HangulTextObservation,
    sampleCount: Int = 25
) -> [HangulOrderedPoint] {
    let count = max(2, sampleCount), box = observation.boundingBox
    if box.height > box.width * 1.4 {
        return (0..

```

4 Experiments

4.1 Datasets and Ground Truth

The experiments are organized into six levels. The Core hard Korean set contains 20 samples: four existing real failure cases and 16 new controlled hard-expansion cases for vertical writing and signboard-like layouts. The Targeted Korean poster synthetic set contains 600 controlled samples with vertical titles, station names,

and multi-column layouts. The manual-template set contains 124 Canva/MiriCanvas poster samples manually transcribed by the user. The public Korean signs set contains 15 samples with manual ground truth. The quantitative metrics are character error rate (CER), word error rate (WER), term recall, and processing time. CER is computed as $ED(norm(\hat{y}), norm(y))/|y|$. CER is not clipped to the range $[0, 1]$. When the ground-truth target is a short title but OCR outputs long surrounding text or digits, insertion errors may exceed the number of ground-truth characters, so CER can be greater than 1.

Table 3: Main experimental and auxiliary probe sets.

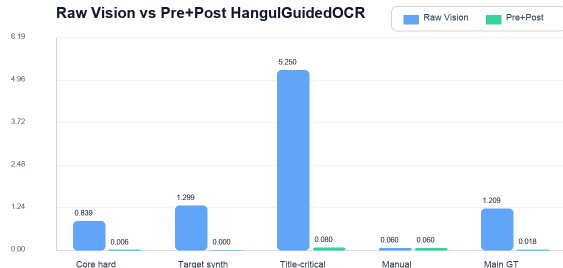
Set	N	Role
Core hard	20	4 real seeds + 16 hard expansion cases
Target synth	600	Controlled vertical title/station/multi-column layouts
Manual templates	124	User-transcribed poster templates
Title-critical	24	Local probe for large title omission
Public signs	15	Public Korean sign samples with manual GT
Stylized title	1	Public-safe title-omission probe

4.2 Evaluation Protocol and Results

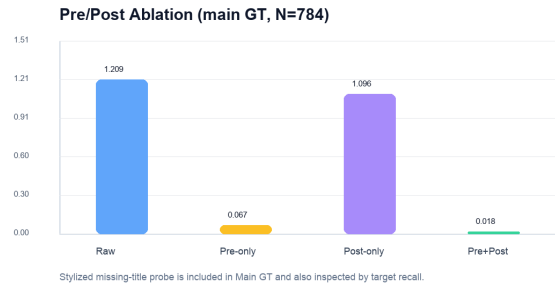
The experiments were conducted on macOS arm64 using Apple Vision `VNRecognizeTextRequest` with `.accurate` recognition level and ko-KR language setting. For the same input, we compared raw, preprocessing-only, postprocessing-only, and preprocessing plus postprocessing auto conditions. Reproducibility materials are split into <https://github.com/wjdalswl/textspotting-guided-vision-ocr> for experiment logs, protocols, and PDFs, and <https://github.com/wjdalswl/HangulGuidedOCR> for the Swift package, summary results, and figures.

Table 4: CER comparison among raw Vision, preprocessing-only, postprocessing-only, and preprocessing plus postprocessing.

Set	N	Raw	Pre	Post	Full
Hard expansion	16	0.947	0.000	0.938	0.000
Real seed	4	0.408	0.028	0.408	0.028
Title-critical	24	5.250	1.680	5.250	0.080
Manual templ.	124	0.060	0.060	0.191	0.060
Public signs	15	0.283	0.283	0.284	0.284
Stylized title	1	14.000	0.333	14.000	0.000
Target synth	600	1.300	0.000	1.130	0.000
Main GT weighted	784	1.209	0.067	1.096	0.018



(a) Main GT CER by OCR variant.



Stylized missing-title probe is included in Main GT and also inspected by target recall.

(b) Raw Vision vs HangulGuidedOCR(auto).

Fig. 3: Quantitative result charts. On the full Main GT aggregate, raw CER was 1.209, pre-only CER was 0.067, post-only CER was 1.096, and pre+post CER was 0.018.

The Main GT weighted value is a sample-count weighted average. Because failure severity differs across subsets, the aggregate CER should be interpreted together with subset-level results. Under a normal approx-

imation, the 95% confidence intervals are 1.209 ± 0.110 for Main GT raw CER, 0.018 ± 0.006 for Main GT full CER, 0.835 ± 0.147 for Core hard raw CER, and 0.005 ± 0.009 for Core hard auto CER. The public-sign subset has only $N = 15$ and is therefore treated as an auxiliary observation.

The claim that Apple Vision OCR is weak on Korean vertical writing and stylized titles is supported by the hard-expansion, title-critical, and stylized-title results in Table 3 and Figs. 4–6. Conversely, the public-sign and manual-template results show that the proposed method does not solve every Korean OCR error.

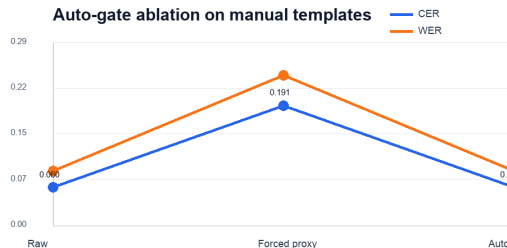


Fig. 4: Comparison between forced proxy reorder and auto decision on ordinary horizontal posters. For ordinary horizontal layouts, the auto strategy should preserve raw order.

4.3 Feedback-driven Open OCR Baseline

The instructor feedback emphasized that Apple Vision OCR alone is a weak baseline structure because it is a closed API, and its internal text-region detection and recognition components cannot be separated. Therefore, this revision explicitly treats Apple Vision as a closed-source black-box OCR baseline and adds Tesseract, EasyOCR, and PaddleOCR as public OCR baselines. DeepSolo is not positioned as a directly comparable performance baseline in this table; it is the text-spotting reference that motivates ordered points and boundary representation.

Table 5: Baseline taxonomy after incorporating the feedback. Apple Vision is a closed OCR API, DeepSolo is a text-spotting representation reference, and HangulGuidedOCR is a correction layer.

Category	Examples	Role in this study
Closed OCR API	Apple Vision OCR	Primary black-box OCR baseline
Open OCR engine	Tesseract, EasyOCR, PaddleOCR	External OCR comparison baseline
Text detector	EAST, CRAFT, DB	Related text-region detection family
Text spotting	TESTR, DeepSolo	Ordered-point/boundary representation reference
Correction layer	HangulGuidedOCR	Proposed Swift/Xcode layer around OCR

Table 6: CER comparison for public OCR baselines. O/G means original/guided strip, and O/S means original/pre-spot strip.

Engine	Scope	GT rows	Target	O/G	Core	O/G	Title	O/S
EasyOCR	full public subset	1236	1.393/0.066	0.840/0.023	9.333/0.333			
Tesseract	full public subset	1236	1.439/0.082	1.255/0.096	6.333/1.667			
PaddleOCR	runtime-bounded subset	36	—/—	0.636/0.459	5.667/1.333			

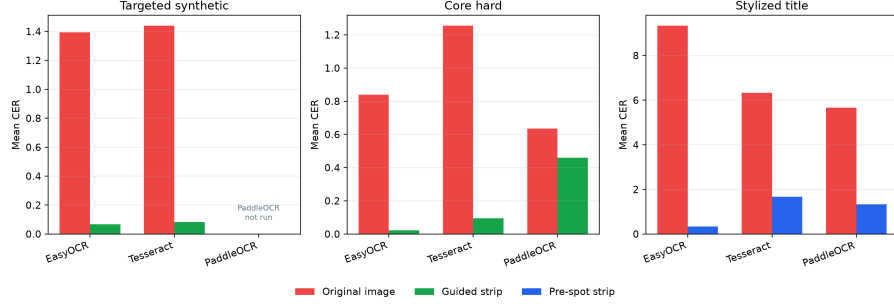


Fig. 5: Comparison with Tesseract, EasyOCR, and PaddleOCR. In targeted synthetic and core hard subsets, guided strip inputs reduce CER compared with full original images. In stylized title cases, the pre-spot strip provides the title region more directly.

This added experiment is not a claim that the internal Apple Vision algorithm was improved. Instead, it narrows the contribution: Apple Vision is a black-box OCR API that may already include text-region detection, while the proposed method structures the input and reading-order representation around OCR. EasyOCR and Tesseract show lower CER for guided strip inputs on targeted synthetic and core hard subsets. PaddleOCR was run only on a runtime-bounded subset, but it follows the same direction on core/title cases. The qualitative contribution is therefore the way candidate regions and order cues are provided for Korean vertical or dispersed text, regardless of the OCR engine.

5 Representative Cases and Discussion

5.1 Vertical Sign and Typography Cases

The Core hard Korean set combines four existing real failure cases and 16 controlled hard poster/sign expansion cases. In the existing real cases, the subway station sign raw output remained incomplete, such as only detecting part of the station label, while dispersed typography was assembled in the wrong order. In the subway case, the final target order was corrected to follow the bilingual sign’s top-to-bottom English cue: “Dongdaemun History” before “Culture Park” in the corresponding Korean order.



(a) Subway vertical station sign recovery.

(b) Dispersed typography recovery.

Fig. 6: Representative conversion cases. Each process arranges the original/ROI, rotation or strip conversion, ordered reading order, and recovered output in one flow.

5.2 Title-Critical Spotting Failure

In the title-critical subset, many Canva/MiriCanvas-style templates contained large titles that were omitted or distorted. If evaluated only by full-poster CER, these failures are diluted by long body text. Therefore, we added a separate target-title evaluation. In this subset, raw CER was 5.250 and post-only CER also remained 5.250, whereas pre+post CER decreased to 0.080. This indicates that for title omissions such as stylized vertical holiday titles, pre-OCR ordered spot proposals are more important than post-OCR reordering alone.

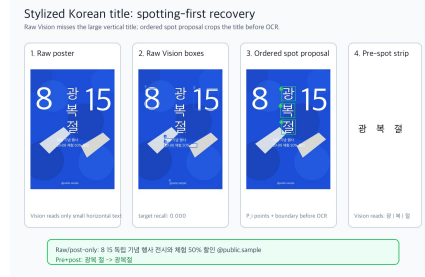


Fig. 7: Recovery process for a stylized vertical title omitted by raw OCR using a pre-OCR ordered spot proposal.

5.3 Discussion

The key point of this experiment is that HangulGuidedOCR must be safe to call all the time, while its internal auto strategy decides when to reorder. In vertical writing and dispersed typography, OCR engines may detect some characters but assemble the string in the wrong order; in those cases, ordered point sequences and boundary priors are effective. In ordinary horizontal posters, however, Vision already preserves paragraph order well, so HangulGuidedOCR(auto) should not apply point-proxy reordering. This is important for real app development: the library must improve difficult vertical or multi-column samples without damaging ordinary posters that already work.

6 Limitations and Future Work

The results of this study should not be interpreted as a direct reproduction of the official DeepSolo model performance. DeepSolo assumes a Python 3.8, Detectron2, and CUDA Linux inference stack, whereas this project was conducted on Apple Silicon macOS, where the official pretrained inference path could not be completed. Therefore, the DeepSolo-related contribution in this report is a representation-proxy reproduction: the ordered points, center-line structure, and boundary cues identified from the paper and public code are transferred onto Apple Vision OCR outputs. This distinction also motivates the revised evaluation framing: Apple Vision OCR is treated as a closed-source black-box baseline, while Tesseract, EasyOCR, and PaddleOCR are added as public OCR-engine baselines.

The dataset also has limitations. The Core hard set contains 20 samples: four real failure seeds and 16 controlled hard-expansion samples. The newly added 16 cases are not independently photographed real-world photos, but controlled samples designed to isolate failure types such as vertical columns, multi-column signs, and dispersed Korean typography. In addition, thresholds such as $xSpread > 0.10$ and height/width ratio are pilot defaults; the current results should be read as evidence of the method’s feasibility rather than as a fully optimized operating point. Future work should collect at least 50–200 balanced real Korean photos across posters, subway signs, shop signs, and public notices, and should add threshold sweeps, bootstrap confidence intervals, and paired significance tests. Canva/MiriCanvas original templates are not redistributed on GitHub because of licensing constraints; the public repository includes reproducible synthetic/public-safe samples and summary results instead.

7 Conclusion

This study investigated whether Apple Vision OCR, a widely used OCR option in iOS and macOS applications, can be corrected for Korean reading-order failures and title omissions without training a new OCR model. The project defines three core failure types: parallel vertical columns in subway station signs, dispersed Korean typography in poster layouts, and stylized large titles such as ‘Gwangbokjeol’ that are missed before recognition. To address these cases, we proposed HangulGuidedOCR, a pipeline that approximates DeepSolo’s ordered-points and boundary representation on top of Apple Vision OCR. The method creates candidate regions before OCR, converts Vision bounding boxes into 25 ordered center-line points with boundary cues, and then automatically decides whether to preserve the raw output or reorder it according to the detected layout structure.

Experimentally, the Main GT weighted CER decreased from 1.209 with raw Vision OCR to 0.018 with HangulGuidedOCR(auto). On the Core hard set, raw CER decreased from 0.835 to 0.005, and on the title-critical probe, CER decreased from 5.250 to 0.080 under the pre+post condition. These results show that post-OCR string reordering alone is insufficient when a large title is missed entirely; pre-OCR ordered spot proposals are necessary to reduce detection failure itself. Conversely, on ordinary horizontal posters and some public sign samples, the auto strategy avoided forced reordering and preserved the raw reading order, which is essential for practical deployment. Additional Tesseract, EasyOCR, and PaddleOCR experiments further suggest that Korean vertical writing and stylized title failures are not limited to one Apple API, but are broader OCR failure modes related to input structure and reading-order estimation.

In conclusion, this work is not a simple performance comparison against DeepSolo. Rather, it applies an ordered representation from end-to-end text spotting research as a practical correction layer around a closed OCR API. The resulting HangulGuidedOCR Swift package does not replace Apple Vision OCR; it lets developers keep their existing Vision pipeline while adding preprocessing and postprocessing support for Korean vertical text, multi-column notices, and dispersed typography. With a larger real-photo benchmark, open text-detector comparisons, and threshold ablation, this approach can mature into a reproducible reading-order correction library for Korean mobile OCR applications.

References

1. Baek, Y., Lee, B., Han, D., Yun, S., Lee, H.: Character region awareness for text detection. In: CVPR Workshops (2019)
2. JaideAI: Easyocr github repository (2020), accessed: 2026-06-08
3. Liao, M., Shi, B., Bai, X., Wang, X., Liu, W.: Textboxes: A fast text detector with a single deep neural network. In: AAAI Conference on Artificial Intelligence (2017)
4. Liao, M., Wan, Z., Yao, C., Chen, K., Bai, X.: Real-time scene text detection with differentiable binarization. In: AAAI Conference on Artificial Intelligence (2020)
5. Liu, Y., Chen, H., Shen, C., He, T., Jin, L., Wang, L.: Abcnet: Real-time scene text spotting with adaptive bezier-curve network. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (2020)
6. NAVER Cloud Platform: Clova ocr product documentation (2026), accessed: 2026-06-08
7. PaddleOCR Contributors: Paddleocr: Awesome multilingual ocr toolkits based on paddlepaddle (2020), accessed: 2026-06-15
8. Shi, B., Bai, X., Yao, C.: An end-to-end trainable neural network for image-based sequence recognition and its application to scene text recognition. IEEE Transactions on Pattern Analysis and Machine Intelligence (2017)
9. Shi, B., Yang, M., Wang, X., Lyu, P., Yao, C., Bai, X.: Aster: An attentional scene text recognizer with flexible rectification. IEEE Transactions on Pattern Analysis and Machine Intelligence (2019)
10. Smith, R.: An overview of the tesseract ocr engine. In: International Conference on Document Analysis and Recognition (2007)
11. Wang, Z., Xu, Y., Cui, L., Shang, J., Wei, F.: Layoutreader: Pre-training of text and layout for reading order detection. In: Proceedings of EMNLP (2021)
12. Ye, M., Zhang, J., Zhao, S., Liu, J., Liu, T., Du, B., Tao, D.: DeepSolo: Let transformer decoder with explicit points solo for text spotting. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (2023)
13. Zhang, X., Su, Y., Tripathi, S., Tu, Z.: Testr: Text spotting transformers. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (2022)
14. Zhou, X., Yao, C., Wen, H., Wang, Y., Zhou, S., He, W., Liang, J.: East: An efficient and accurate scene text detector. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (2017)